

BUỔI 2. GIẢI THUẬT K LÁNG GIỀNG GẦN NHẤT

Mục đích:

- Củng cố lý thuyết và có thể cài đặt giải thuật KNN, Bayes thơ ngây.
- Kiểm thử và đánh giá kết quả sau khi huấn luyện mô hình.

2.1. Cài đặt thư viện

Để cài đặt được các giải thuật, việc đầu tiên chúng ta phải làm chính là cài đặt các thư viện của Python để hỗ trợ cho việc cài đặt mô hình. Ở đây chúng ta dùng thư viện **scikit-learn**.

```
pip install -U scikit-learn
```

Câu lệnh này có thể được chạy trong bảng lệnh command line trên máy tính hoặc chạy trên terminal nếu sử dụng công cụ lập trình Pycharm.

2.2. KNN cho bài toán phân loại

2.2.1. Phân tích tập dữ liệu

Sau khi cài đặt thư viện, chúng ta tiến hành xây dựng mô hình máy học trên tập dữ liệu mẫu rất phổ biến đó là tập dữ liệu **Iris**: <https://www.kaggle.com/datasets/uciml/iris>

Bộ dữ liệu này bao gồm thông tin của ba loại hoa Iris (một loài hoa lan) khác nhau: Iris setosa, Iris virginica và Iris versicolor. Mỗi loại có 50 bông hoa được đo với dữ liệu là 4 thông tin: chiều dài, chiều rộng đài hoa (sepal), và chiều dài, chiều rộng cánh hoa (petal). Tổng số dữ liệu là 150 cho 3 lớp hoa.

Ở phần minh họa này, chúng ta sẽ sử dụng nghi thức hold-out để kiểm tra độ chính xác bằng cách chia tập dữ liệu ra thành hai phần (2/3 cho tập huấn luyện và 1/3 cho tập kiểm tra).

Đầu tiên, chúng ta cần load dữ liệu vào và hiển thị lên các thông tin về tập dữ liệu này để có cái nhìn trực quan về nó. Ta có thể đọc dữ liệu bằng 2 cách sau:

- Cách 1. Đọc tập dữ liệu từ file csv:

Sử dụng read_csv của thư viện pandas để đọc dữ liệu từ file CSV.

```

1 ✓ from pandas import read_csv
2 import numpy as np
3 # 1. Load dữ liệu Iris từ file CSV
4 iris = read_csv('Iris.csv', delimiter=',', index_col=0)
5
6 # 2. Lấy dữ liệu thuộc tính lưu vào data_X
7 iris_X = iris.iloc[:, :-1]
8
9 # 3. Lấy dữ liệu nhãn lưu vào data_y
10 iris_y = iris.iloc[:, -1]
11
12 # 4. Chuyển đổi dữ liệu nhãn thành dạng số
13 iris_y = iris_y.map({'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-virginica': 2})
14
15 # 5. In các thông tin về nhãn và số lượng dữ liệu
16 print("Number of classes: ", len(np.unique(iris_y)))
17 print("Number of data: ", len(iris_y))
18 print()
19
20 # 6. Hiển thị 5 hàng dữ liệu mẫu của mỗi nhãn
21 ✓ for i in range(3):
22     print(f"Sample data from class {i}:")
23     print(iris_X[iris_y == i].head())
24     print()

```

Kết quả sau khi thực thi:

```

Number of classes: 3
Number of data: 150

Sample data from class 0:
      SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Id
1           5.1           3.5           1.4           0.2
2           4.9           3.0           1.4           0.2
3           4.7           3.2           1.3           0.2
4           4.6           3.1           1.5           0.2
5           5.0           3.6           1.4           0.2

Sample data from class 1:
      SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Id
51           7.0           3.2           4.7           1.4
52           6.4           3.2           4.5           1.5
53           6.9           3.1           4.9           1.5
54           5.5           2.3           4.0           1.3
55           6.5           2.8           4.6           1.5

Sample data from class 2:
      SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Id
101           6.3           3.3           6.0           2.5
...
103           7.1           3.0           5.9           2.1
104           6.3           2.9           5.6           1.8
105           6.5           3.0           5.8           2.2

```

Cách 2. Đọc tập dữ liệu từ thư viện sklearn:

Sử dụng các hàm hỗ trợ của thư viện sklearn đọc tập dữ liệu Iris.

```
import numpy as np
from sklearn import neighbors, datasets
# Load dữ liệu Iris
iris = datasets.load_iris()
# Lấy dữ liệu thuộc tính lưu vào biến iris_X
iris_X = iris.data
# Lấy dữ liệu nhãn lưu vào biến iris_y
iris_y = iris.target
# In ra các thông tin về nhãn và số lượng dữ liệu
print("Number of classes: %d" % len(np.unique(iris_y)))
print("Number of data points: %d" % len(iris_y))

# Hiển thị 5 dữ liệu có nhãn 0
x0 = iris_X[iris_y == 0, :]
print("Samples from class 0:\n", x0[:5, :])
# Hiển thị 5 dữ liệu có nhãn 1
x1 = iris_X[iris_y == 1, :]
print("Samples from class 1:\n", x1[:5, :])
# Hiển thị 5 dữ liệu có nhãn 2
x2 = iris_X[iris_y == 2, :]
print("Samples from class 2:\n", x2[:5, :])
```

Kết quả sau khi thực thi như sau:

```
C:\Users\PC\PycharmProjects\TH_Buoi01\venv\Scripts\python.exe C:\Users\PC\Pycha
Number of classes: 3
Number of data points: 150
Samples from class 0:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
Samples from class 1:
[[7.  3.2 4.7 1.4]
 [6.4 3.2 4.5 1.5]
 [6.9 3.1 4.9 1.5]
 [5.5 2.3 4.  1.3]
 [6.5 2.8 4.6 1.5]]
Samples from class 2:
[[6.3 3.3 6.  2.5]
 [5.8 2.7 5.1 1.9]
 [7.1 3.  5.9 2.1]
 [6.3 2.9 5.6 1.8]
 [6.5 3.  5.8 2.2]]
```

2.2.2. Cài đặt giải thuật KNN cho bài toán phân loại

Sau khi khai phá được tập dữ liệu, chúng ta thực hiện chia tập dữ liệu thành hai phần theo nghi thức kiểm tra hold-out đã đề cập bên trên. Việc chia được thực hiện bằng đoạn code sau:

```
from sklearn.model_selection import train_test_split
# Phân chia tập dataset
X_train, X_test, y_train, y_test = train_test_split(
    iris_X, iris_y, test_size=50)

print("Training size: %d" % len(y_train))
print("Test size      : %d" % len(y_test))
print(y_test)
```

Kết quả là chúng ta sẽ có được 2 tập dữ liệu:

+ Tập dữ liệu huấn luyện (Training dataset) gồm 100 phần tử.

+ Tập dữ liệu kiểm tra (Testing dataset) gồm 50 phần tử.

Tiếp theo, Chúng ta cần tìm hiểu các tham số khi xây dựng mô hình huấn luyện KNN, sử dụng hàm **KneighborsClassifier()**.

Đây là hàm mà thư viện cung cấp sẵn cho chúng ta sử dụng nên việc chúng ta cần làm là khám phá các tham số đầu vào của hàm này để tinh chỉnh cho phù hợp với tập dữ liệu của chúng ta. Các tham số từ hàm này có thể kể lần lượt như sau:

+ `n_neighbors` – Số lượng láng giềng mà chúng ta mong muốn

+ `weights` – Trọng số mà chúng ta sử dụng kết hợp khi dự đoán kết quả. Các giá trị có thể cấu hình là:

- 'uniform' : Tất cả các điểm dữ liệu sẽ có độ quan trọng (hay trọng số) như nhau.
- 'distance' : trọng số được tính bằng cách nghịch đảo khoảng cách. Trong trường hợp này, phần tử nào càng nằm gần phần tử dự đoán thì trọng số sẽ càng cao.
- [callable] : người dùng có thể định nghĩa hàm tính trọng số cho riêng bài toán mà mình mong muốn.

+ `algorithm` – giải thuật dùng để tính toán kết quả dự đoán. Có các giá trị sau:

- 'ball_tree' nếu sử dụng BallTree
- 'kd_tree' nếu sử dụng KDTree
- 'brute' nếu sử dụng tìm kiếm vét cạn.
- 'auto' lựa chọn giải thuật phù hợp với dữ liệu hiện tại đang huấn luyện.

+ `leaf_size` – sử dụng cho các giải thuật liên quan đến cây BallTree hoặc KDTree.

+ `p` – bậc trong công thức Minkowski. Khi $p = 1$, thì sẽ thành khoảng cách Manhattan (L1), và khoảng cách Euclidean (L2) khi $p = 2$.

+ `metric` – độ đo được sử dụng cho việc tính khoảng cách.

- Mặc định là "minkowski", và kết quả sẽ trả về dựa vào p .

- Độ đo là "precomputed", X được coi là ma trận khoảng cách và phải vuông góc khi khớp. X có thể là một biểu đồ thưa thớt, trong trường hợp đó chỉ các phần tử "khác 0" mới có thể được coi là lân cận.

Và một số tham số khác như `metric_params`, `n_jobs`

Sau khi tìm hiểu các tham số, chúng ta tiến hành xây dựng mô hình KNN cho bài toán phân loại thông qua hàm **KneighborsClassifier()** bằng đoạn code sau

```
# Huấn luyện mô hình KNN
model_KNN = neighbors.KNeighborsClassifier(n_neighbors=3, p=2)
model_KNN.fit(X_train, y_train)
# Dự đoán kết quả trả về từ tập test
y_pred = model_KNN.predict(X_test)
# Hiển thị kết quả của 20 dữ liệu test
print("Print results for 20 test data points:")
print("Predicted labels: ", y_pred[0:20])
print("Ground truth   : ", y_test[0:20])
```

2.2.3. Kiểm thử và đánh giá kết quả

Sau khi thực hiện huấn luyện mô hình KNN, chúng ta sẽ đến với việc kiểm tra kết quả.

Kết quả đầu tiên mà chúng ta cần phải xác định đó là độ chính xác của mô hình huấn luyện. Chúng ta sẽ đo độ chính xác dựa trên tập kiểm tra qua đoạn code sau:

```
from sklearn.metrics import accuracy_score
# Tính độ chính xác tổng thể
print("Accuracy of KNN: %.2f %" % (100 * accuracy_score(y_test, y_pred)))
```

Chương trình sẽ cho ra độ chính xác của mô hình huấn luyện.

Ngoài ra, chúng ta còn có thể hiển thị ma trận phân lớp cho mô hình sau khi huấn luyện bằng việc kiểm tra kết quả trên tập Test qua đoạn code sau:

```

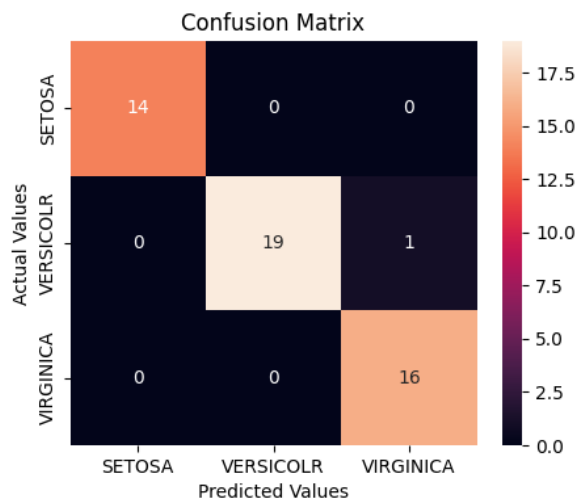
# Xây dựng ma trận phân lớp
# Cách 01:
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred, labels=[2, 0, 1])
print(cm)

# Cách 02:
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, y_pred, labels=[0, 1, 2])
cm_df = pd.DataFrame(cm, index=['SETOSA', 'VERSICOLR', 'VIRGINICA'],
                      columns=['SETOSA', 'VERSICOLR', 'VIRGINICA'])
# Hiển thị ma trận phân lớp
plt.figure(figsize=(5, 4))
sns.heatmap(cm_df, annot=True)
plt.title('Confusion Matrix')
plt.ylabel('Actual Values')
plt.xlabel('Predicted Values')
plt.show()

```

Kết quả ma trận phân lớp như sau:



Tính độ chính xác trên từng lớp

Ma trận phân lớp cung cấp thông tin về số lượng mẫu dự đoán đúng và sai cho từng lớp có thể được biểu diễn như sau:

Dự đoán =>	1	...	k
1			
...			
k			

$C[i,j]$: số phần tử lớp i (dòng) được giải thuật dự đoán là lớp j (cột)

$C[i,i]$: số phần tử phân lớp đúng

Độ chính xác trên từng lớp có thể tính theo công thức sau:

$$\text{Độ chính xác của lớp } i = \frac{C[i,i]}{C[i,]}$$

```
def calculate_accuracy_per_class(conf_matrix):
    accuracies = []
    for i in range(len(conf_matrix)):
        correct_predictions = conf_matrix[i, i]
        total_predictions = np.sum(conf_matrix[i, :])
        accuracy = correct_predictions / total_predictions if total_predictions != 0 else 0
        accuracies.append(accuracy)
    return accuracies

accuracies = calculate_accuracy_per_class(con_matrix)

# Hiển thị độ chính xác của từng lớp
for i, accuracy in enumerate(accuracies):
    print(f"Accuracy of class {i} ({con_matrix_df.index[i]}): {accuracy:.2f}")

Accuracy of class 0 (SETOSA): 1.00
Accuracy of class 1 (VERSICOLR): 0.94
Accuracy of class 2 (VIRGINICA): 0.94
```

* Bài tập thực hành vận dụng trên lớp

Bài 01: Hãy tạo file thống kê (có thể word hoặc excel) có tên “ThongKeKQ_KNN” nằm trong thư mục “TH_Buoi02_<HoTen>_<MSSV>” chứa các kết quả kiểm tra từ việc xây dựng mô hình KNN trên tập dữ liệu Iris theo yêu cầu như sau:

- Về phân chia tập dữ liệu:

+ Nghi thức kiểm tra hold-out (2/3 cho tập Train, 1/3 cho tập Test)

+ Chia tập dữ liệu thành 80% cho tập Train, 20% cho tập Test

- Về chọn tham số k:

Ứng với mỗi cách phân chia dữ liệu bên trên, kèm với mỗi độ đo khoảng cách là Manhattan và Euclide. Hãy xây dựng mô hình KNN với các tham số k lần lượt là 1, 2, 4, 6, 8, 10.

Với mỗi giá trị k, hãy tính độ chính xác trên tập kiểm tra. Sau đó vẽ biểu đồ mô tả độ chính xác theo tham số k.

Mỗi trường hợp đều xuất ra ma trận phân lớp tương ứng.

- Cuối cùng, hãy trả lời câu hỏi sau: Anh/Chị hãy cung cấp một bộ tham số huấn luyện mô hình KNN cho tập dữ liệu Iris mà theo Anh/Chị là tốt nhất?

2.3. Chuẩn hóa dữ liệu (min-max scaling)

Các thuật toán sẽ dựa trên khoảng cách giữa các điểm dữ liệu, nếu các đặc trưng của các điểm dữ liệu có các đơn vị hoặc tỷ lệ khác nhau, điều này có thể dẫn đến kết quả không chính xác. Để khắc phục vấn đề này và đảm bảo kết quả phân tích chính xác hơn, người ta thường sử dụng kỹ thuật chuẩn hóa dữ liệu (Data normalization). Ở bài thực hành này chúng ta sẽ sử dụng phương pháp chuẩn hóa dữ liệu là **Min-Max Scaling** đưa tất cả các đặc trưng về cùng một khoảng giá trị, thường là từ (0;1) hoặc (-1;1), trước khi thực hiện các thuật toán. Phương pháp này có công thức sau:

$$new_val = \frac{val - min}{max - min}$$

Trong đó, min và max là giá trị nhỏ nhất và lớn nhất của đặc trưng tương ứng.

Ví dụ minh họa phương pháp chuẩn hóa dữ liệu **Min-Max Scaling** sử dụng hàm **MinMaxScaler()** được thư viện sklearn cung cấp. Chuẩn hóa trên tập dữ liệu đánh giá chất lượng rượu vang **đỏ** trên trang UCI:

<https://archive.ics.uci.edu/ml/datasets/wine+quality>

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('/content/winequality-red.csv', delimiter=';')

# Tách các đặc trưng(features) và nhãn(target) của dữ liệu
x = df.drop('quality', axis=1)
y = df['quality']

# Khởi tạo hàm MinMaxScaler
scaler = MinMaxScaler()

# Chuyển đổi các đặc trưng
X_scaler = scaler.fit_transform(x)

# Tạo DataFrame và in ra các đặc trưng đã chuẩn hóa
df_scaler = pd.DataFrame(X_scaler, columns = x.columns)
print("Dữ liệu trước khi chuẩn hóa:\n", df.head())
print("Dữ liệu sau khi chuẩn hóa:\n", df_scaler.head())
```

Kết quả sau khi thực thi:

```

Dữ liệu trước khi chuẩn hóa:
fixed acidity volatile acidity citric acid residual sugar chlorides \
0      7.4          0.70      0.00      1.9      0.076
1      7.8          0.88      0.00      2.6      0.098
2      7.8          0.76      0.04      2.3      0.092
3     11.2          0.28      0.56      1.9      0.075
4      7.4          0.70      0.00      1.9      0.076

free sulfur dioxide total sulfur dioxide density pH sulphates \
0      11.0          34.0      0.9978 3.51      0.56
1      25.0          67.0      0.9968 3.20      0.68
2      15.0          54.0      0.9970 3.26      0.65
3      17.0          60.0      0.9980 3.16      0.58
4      11.0          34.0      0.9978 3.51      0.56

alcohol quality
0      9.4          5
1      9.8          5
2      9.8          5
3      9.8          6
4      9.4          5

Dữ liệu sau khi chuẩn hóa:
fixed acidity volatile acidity citric acid residual sugar chlorides \
0      0.247788      0.397260      0.00      0.068493 0.106845
1      0.283186      0.520548      0.00      0.116438 0.143573
2      0.283186      0.438356      0.04      0.095890 0.133556
3      0.584071      0.109589      0.56      0.068493 0.105175
4      0.247788      0.397260      0.00      0.068493 0.106845

free sulfur dioxide total sulfur dioxide density pH sulphates \
0      0.140845      0.098940 0.567548 0.606299 0.137725
1      0.338028      0.215548 0.494126 0.362205 0.209581
2      0.197183      0.169611 0.508811 0.409449 0.191617
3      0.225352      0.190813 0.582232 0.330709 0.149701
4      0.140845      0.098940 0.567548 0.606299 0.137725

alcohol
0 0.153846
1 0.215385
2 0.215385
3 0.215385
4 0.153846

```

Hãy tạo thư mục tên “TH_Buoi02_<HoTen>_<MSSV>”, sau đó tạo các tập tin cần thiết cho bài tập thực hành sau đây:

2.4. KNN cho bài toán hồi quy

2.4.1. Phân chia tập dữ liệu

Chúng ta sử dụng tập dữ liệu *california_housing* do thư viện *sklearn* cung cấp.

Bộ dữ liệu này có 20.640 mẫu, gồm các đặc trưng sau:

- **MedInc**: Thu nhập trung bình của các hộ gia đình trong khu vực (nghìn USD).
- **HouseAge**: Tuổi trung bình của nhà cửa trong khu vực.

- **AveRooms:** Số lượng phòng trung bình trong mỗi ngôi nhà.
- **AveBedrms:** Số lượng phòng ngủ trung bình trong mỗi ngôi nhà.
- **Population:** Dân số trong khu vực.
- **AveOccup:** Số lượng người ở trung bình trong mỗi ngôi nhà.
- **Latitude:** Vĩ độ của khu vực.
- **Longitude:** Kinh độ của khu vực.

Và biến **target** là giá trị nhà trung bình các quận của California, được thể hiện bằng hàng trăm nghìn đô la (100.000 đô la).

Ở phần minh họa này, chúng ta sẽ sử dụng nghi thức hold-out để kiểm tra độ chính xác bằng cách chia tập dữ liệu ra thành hai phần (2/3 cho tập huấn luyện và 1/3 cho tập kiểm tra).

Đầu tiên, chúng ta cần đọc dữ liệu vào. Có thể thực hiện bởi đoạn code sau:

```
# Import các thư viện cần thiết
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

# Load dữ liệu California Housing
data = fetch_california_housing()

# X là các đặc trưng (features), y là nhãn (target) - giá nhà trung bình
X = data.data
y = data.target

# Chia dữ liệu thành tập huấn luyện (70%) và tập kiểm tra (30%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

2.4.2. Cài đặt giải thuật KNN cho bài toán hồi quy

Sau khi phân chia tập dữ liệu, chúng ta tiến hành xây dựng mô hình KNN cho nhiệm vụ hồi quy thông qua hàm **KneighborsRegressor** bằng đoạn code sau:

Đây là hàm mà thư viện cung cấp sẵn cho chúng ta sử dụng nên việc chúng ta cần làm là khám phá các tham số đầu vào của hàm này để tinh chỉnh cho phù hợp với tập dữ liệu của chúng ta. Các tham số từ hàm này có thể kể lần lượt như sau:

+ `n_neighbors` – Số lượng láng giềng mà chúng ta mong muốn

+ `weights` – Trọng số mà chúng ta sử dụng kết hợp khi dự đoán kết quả. Các giá trị có thể cấu hình là:

- `'uniform'` : Tất cả các điểm dữ liệu sẽ có độ quan trọng (hay trọng số) như nhau.
- `'distance'` : trọng số được tính bằng cách nghịch đảo khoảng cách. Trong trường hợp này, phần tử nào càng nằm gần phần tử dự đoán thì trọng số sẽ càng cao.
- `[callable]` : người dùng có thể định nghĩa hàm tính trọng số cho riêng bài toán mà mình mong muốn.

+ `algorithm` – giải thuật dùng để tính toán kết quả dự đoán. Có các giá trị sau:

- `'ball_tree'` nếu sử dụng `BallTree`
- `'kd_tree'` nếu sử dụng `KDTree`
- `'brute'` nếu sử dụng tìm kiếm vét cạn.
- `'auto'` lựa chọn giải thuật phù hợp với dữ liệu hiện tại đang huấn luyện.

+ `leaf_size` – sử dụng cho các giải thuật liên quan đến cây `BallTree` hoặc `KDTree`.

+ `p` – bậc trong công thức Minkowski.

$$d(x, y) = \sqrt[p]{\sum_{i=1}^n |x_i - y_i|^p}$$

Trong đó:

- $p \geq 1$
- $x = (x_1, x_2, \dots, x_i)$ và $y = (y_1, y_2, \dots, y_i)$.

- Khi $p = 1$, thì sẽ thành khoảng cách Manhattan (L1)

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

- Khi $p = 2$, d là khoảng cách Euclid (L2)

$$d(x, y) = \sqrt{\sum_{i=1}^n |x_i - y_i|^2}$$

+ metric – độ đo được sử dụng cho việc tính khoảng cách.

- Mặc định là "minkowski", và kết quả sẽ trả về dựa vào p .
- Độ đo là "precomputed", X được coi là ma trận khoảng cách và phải vuông góc khi khớp. X có thể là một biểu đồ thưa thớt, trong trường hợp đó chỉ các phần tử "khác 0" mới có thể được coi là lân cận.

Và một số tham số khác như `metric_params`, `n_jobs`

```
# Khởi tạo KNN cho hồi quy với k = 3
model_KNN_regressor = neighbors.KNeighborsRegressor(n_neighbors=3, p=2)

# Huấn luyện mô hình
model_KNN_regressor.fit(X_train, y_train)

# Dự đoán trên tập kiểm tra
y_pred = model_KNN_regressor.predict(X_test)
```

2.2.3. Kiểm thử và đánh giá kết quả

Sau khi thực hiện huấn luyện mô hình KNN cho bài toán hồi quy, chúng ta sẽ tiến hành kiểm tra kết quả. Kết quả chúng ta cần phải xác định là:

- MAE: Trung bình sai số tuyệt đối giữa giá trị thực và giá trị dự đoán

- MSE: Trung bình bình phương sai số giữa giá trị thực và giá trị dự đoán
- RMSE: độ lệch chuẩn của các sai số giữa giá trị thực và giá trị dự đoán

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
# Đánh giá mô hình
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(f"Mean Absolute Error: {mae}")
print(f"Mean Squared Error: {mse}")
print(f"Root Mean Square Error: {rmse}")
```

```
Mean Absolute Error: 0.830772720176572
Mean Squared Error: 1.1812604490550584
Root Mean Square Error: 1.086858062975593
```

2.5. Bài tập tổng hợp

Hãy tạo thư mục tên “BaiTapTongHop” nằm bên trong thư mục “TH_Buoi02_<HoTen>_<MSSV>”, sau đó tạo các tập tin (với đuôi mở rộng .py) ứng với từng bài tập vận dụng sau đây:

Bài 01: Đọc tập dữ liệu "Boston housing" theo link sau: <https://www.kaggle.com/datasets/altavish/boston-housing-dataset> để xây dựng mô hình KNN hồi quy dự đoán giá trị trung bình của các ngôi nhà trong những khu vực đó. Thực hiện các bước sau:

- Chuẩn hóa dữ liệu và phân chia tập dữ liệu thành 80% cho tập huấn luyện và 20% cho tập kiểm tra.
- Xây dựng mô hình KNN hồi quy với các giá trị k khác nhau (ví dụ: 1, 3, 5, 7, 9).
- Đánh giá mô hình dựa trên MAE, MSE và RMSE.
- So sánh độ sai số của mô hình trước và sau khi chuẩn hóa dữ liệu.

Bài 02:

- Đọc dữ liệu từ tập dữ liệu đánh giá chất lượng rượu vang **đỏ** trên trang UCI:

<https://archive.ics.uci.edu/ml/datasets/wine+quality>

b. Phân tích tập dữ liệu như ví dụ minh họa ở phần 2.2

c. Sử dụng 4 phần để xây dựng mô hình và sử dụng 1 phần để thực hiện đánh giá mô hình. Anh/chị ghi chú lại số lượng phần tử trong tập test và các giá trị nhãn khác nhau của các phần tử thuộc tập test trong file code.

d. Xây dựng mô hình KNN với 9 láng giềng dựa trên tập dữ liệu học tạo ra ở câu c.

- Đánh giá độ chính xác tổng thể và độ chính xác của từng lớp cho toàn bộ dữ liệu trong tập test.
- In ra giá trị nhãn dự đoán và thực tế của 7 phần tử đầu tiên trong tập test. Đánh giá độ chính xác tổng thể 7 phần tử đầu tiên trong tập test.
- In ra ma trận phân lớp của tập kiểm tra và tính độ chính xác trên từng lớp.
- So sánh độ chính xác của mô hình KNN trước và sau khi chuẩn hóa dữ liệu.

Bài 03:

a. Đọc dữ liệu từ tập dữ liệu đánh giá chất lượng rượu vang **trắng** trên trang UCI:

<https://archive.ics.uci.edu/ml/datasets/wine+quality>

b. Phân tích tập dữ liệu như ví dụ minh họa ở phần 2.2

c. Sử dụng 8 phần để xây dựng mô hình và sử dụng 2 phần để thực hiện đánh giá mô hình. Anh/chị ghi chú lại số lượng phần tử trong tập test và các giá trị nhãn khác nhau của các phần tử thuộc tập test trong file code.

d. Xây dựng mô hình KNN với 7 láng giềng dựa trên tập dữ liệu học tạo ra ở câu c.

- Đánh giá độ chính xác tổng thể và độ chính xác của từng lớp cho toàn bộ dữ liệu trong tập test.
- In ra giá trị nhãn dự đoán và thực tế của 8 phần tử đầu tiên trong tập test. Đánh giá độ chính xác tổng thể 8 phần tử đầu tiên trong tập test.
- In ra ma trận phân lớp của tập kiểm tra và tính độ chính xác trên từng lớp.
- So sánh độ chính xác của mô hình KNN trước và sau khi chuẩn hóa dữ liệu.

() Tất cả bài tập ở buổi 2 cần phải được lưu trữ theo đúng cấu trúc cây thư mục mà bài thực hành yêu cầu. Sinh viên sẽ nén thư mục gốc lại và nộp cho giảng viên theo thời hạn mà giảng viên yêu cầu.*